📚 CIM-PIM Transformation Tool

Générateur Automatique de Documentation à partir de modèles conceptuels (XMI) et configurations structurées.



🚀 Fonctionnalités

- **Génération Automatique** de documentation Markdown professionnelle
- **Support Multi-Sources** : XMI, YAML, JSON, CSV
- **Sections Complètes** : Page de garde, structure et dynamiques, Initialisation, indicateurs, mode d'emploi
- **Configuration Interactive** : Assistant de configuration pas-à-pas
- **Validation des Données** : Schémas intégrés pour la qualité
- **Export Personnalisable** : Templates modulables

📦 Installation

Prérequis Système

- **Python 3.8+** : Télécharger Python
- **Git** (optionnel) : Télécharger Git

Installation Rapide

```
# Cloner le repository (si versionné)
git clone https://github.com/tingourosanogo/Model_Documentation_Generator.git
cd Model_Documentation_Generator

# Ou télécharger les fichiers directement
# Puis lancer le guide d'installation
./setup_guide.sh
```

Installation Manuelle

```
# 1. Installer les dépendances Python
pip install lxml pyyaml questionary

# 2. Créer la structure de dossiers
mkdir -p input output assets schemas

# 3. Configurer manuellement le config.json

# 4. Configurer manuellement ou utiliser le guide
python main.py
```

🎯 Guide de Démarrage Rapide

Pour les Nouveaux Utilisateurs

1. Lancez l'assistant :

```
./setup_guide.sh
```

2. Suivez les étapes interactives :

- ✅ Installation des dépendances
- 📝 Configuration des métadonnées
- 👥 Ajout des auteurs
- 🏗️ Configuration du projet
- 📁 Choix des fichiers d'entrée
- 📁 Choix du fichier de sortie

3. Générez la documentation :

```
python main.py
```

📁 Structure du Projet

```
Model_Documentation_Generator/
├── src/                          # Code source
│   ├── sections/                 # Sections de documentation
│   │   ├── coverPage.py          # Page de garde
│   │   ├── introduction.py       # Introduction
│   │   ├── structure.py          # Structure et dynamiques
│   │   ├── initialization.py     # Initialization
│   │   ├── indicators.py         # Indicateurs
│   │   └── userManual.py         # Mode d'emploi
│   ├── parsers/                                  # Analyseurs de fichiers
│   │   ├── xmi_parser.py                         # Parser XMI/UML
│   │   ├── parser_csv_to_json_indicator.py       # Parser CSV/JSON
│   │   ├── parser_csv_to_json_initialization.py  # Parser CSV/JSON
│   │   └── yaml_to_json.py                       # Parser YAML/JSON
│   ├── loader/                                   # Téléchargeurs de fichiers
│   │   ├── introduction_loader.py                # Loader Introduction
│   │   ├── initialization_loader.py              # Loader Initialization
│   │   ├── indicator_loader.py                   # Loader Indicateurs
│   │   └── userManual_loader.py                  # Loader Mode d'emploi
```

```
│   └── doc_builder.py         # Générateur principal
├── input/                     # Fichiers d'entrée
│   ├── introduction.yml       # Section introduction (YAML/JSON)
│   ├── modele_conceptuel.xmi  # Modèle XMI
│   ├── initialisation.csv     # Initialisation (YAML/Multi-CSV/JSON)
│   ├── indicators.csv         # Indicateurs (YAML/CSV/JSON)
│   ├── user_manual.yml        # Mode d'emploi (YAML/JSON)
│   ├── initialization_data/       # Fichiers CSV (Multi-CSV)
│   │   ├── metadata_general.csv
│   │   ├── metadata_authors.csv
│   │   ├── metadata_references.csv
│   │   ├── global_config.csv
│   │   ├── input_data.csv
│   │   ├── entities.csv
│   │   ├── parameters.csv
│   │   ├── state_variables.csv
│   │   ├── scenarios.csv
│   │   ├── traceability.csv
│   │   ├── validation_methods.csv
│   │   └── initialization_rules.csv
├── output/               # Documentation générée
│   └── documentation.md  # Fichier de sortie
├── exports/
│   ├── documentation.html        # Version web (avec CSS)
│   ├── documentation.pdf         # Version imprimable (PDF)
│   └── documentation.docx        # Version Word (éditable)
├── assets/              # Ressources
│   ├── logo.png         # Logo du projet
│   ├── cover.jpg        # Image de couverture
│   └── custom.css               # CSS personnalisé (optionnel)
├── schemas/             # Schémas de validation
│   ├── indicators_schema.json
│   └── parameters_schema.yml
├── config.json          # Configuration principale
├── setup_guide.sh       # Assistant d'installation
└── main.py             # Point d'entrée
```

⚙️ Configuration Détaillée

Format du Fichier config.json

```
{
  "metadata": {
    "title": "Titre de la documentation",
    "subtitle": "Sous-titre explicatif",
    "version": "1.0.0",
    "status": "Brouillon/Revue/Final",
    "classification": "Interne/Public/Confidentiel",
    "language": "fr",
    "keywords": ["mot-clé1", "mot-clé2"]
  },
```

```
  "authors": [
    {
      "name": "Prénom Nom",
      "affiliation": "Institution",
      "role": "Rôle dans le projet",
      "email": "email@institution.fr",
      "orcid": "0000-0000-0000-0000"
    }
  ],
  "project": {
    "name": "Nom du projet",
    "acronym": "ACRONYME",
    "funding": "Organisme financeur",
    "grant_number": "NUMERO_SUBVENTION",
    "duration": "2024-2026",
    "website": "https://projet.org",
    "repository": "https://github.com/organisation/projet"
  },
  "documentation": {
    "type": "Type de document",
    "purpose": "Objectif de la documentation",
    "audience": "Public cible",
    "conformance": "Standard suivi",
    "license": "Licence de diffusion"
  },
  "technical": {
    "model_version": "Version du modèle",
    "platform": "Plateforme technique",
    "programming_language": "Langage de programmation",
    "minimum_ram": "8GB",
    "recommended_ram": "16GB",
    "dependencies": ["Python 3.8", "Java 11"],
    "data_formats": ["CSV", "JSON", "XMI"]
  },
  "paths": {
    "output_path": "chemin/vers/output/documentation.md",
    "Introduction": {
      "input_path": "chemin/vers/input/introduction.yml",
      "json_path": "chemin/vers/input/introduction.json",
      "format": "yaml",
      "delimitor": ""
    }
  },
  "assets": {
    "logo": "chemin/vers/assets/logo.png",
    "cover": "chemin/vers/assets/cover.jpg"
  },
  "schemas": {
    "indicators_schema": "chemin/vers/schemas/indicators_schema.json"
  }
}
```

📊 Formats de Fichiers Supportés

Introduction

- **YAML (.yml)** : Structure hiérarchique lisible
- **JSON (.json)** : Format standard interchangeable

Structure

- **XMI (.xmi)** : Format standard UML pour modèles conceptuels

Indicateurs

- **YAML (.yml)** : Pour configurations complexes
- **JSON (.json)** : Pour intégration avec d'autres outils
- **CSV (.csv)** : Pour données tabulaires simples

Initialisation

- **Multi-CSV (dossier/)** : Multiple fichiers CSV organisés
- **YAML (.yml)** : Configuration structurée
- **JSON (.json)** : Données d'initialisation

Mode d'Emploi

- **YAML (.yml)** : Instructions structurées
- **JSON (.json)** : Données de procédure

🛠️ Utilisation Avancée

- **API Python**

```
from src.builder import DocumentationBuilder
from src.config import load_config
from src.parsers.xmi_parser import XMIParser

# Chargement configuration
config = load_config("config.json")

# Parsing manuel
parser = XMIParser("input/modele.xmi")
classes = parser.get_classes()

# Génération personnalisée
builder = DocumentationBuilder(config)
documentation = builder.build()

# Export vers fichier
```

```
with open("output/docs.md", "w", encoding="utf-8") as f:
    f.write(documentation)
```

- **ligne de Commande**

```
# Génération simple
python main.py

# Avec configuration spécifique
python main.py --config mon_config.json --output ma_documentation.md

# Génération partielle (sections spécifiques)
python main.py --sections cover,structure --output partial_docs.md

# Mode verbose pour le débogage
python main.py --verbose --debug

# Générer seulement Markdown
python main.py --config config.json

# Générer Markdown + HTML
python main.py --export html

# Générer Markdown + HTML + PDF
python main.py --export html pdf

# Avec CSS personnalisé
python main.py --export html pdf --css assets/custom.css

# Spécifier le dossier d'export
python main.py --export html pdf docx --export-dir docs/export

# Générer seulement PDF
python main.py --export pdf

# Générer seulement certaines section
python main.py --sections Introduction Indicateurs
```

Arguments Principaux `--config` : Fichier de configuration personnalisé

`--output` : Chemin de sortie personnalisé

`--sections` : Génération partielle par sections

`--export` : Export multi-format (HTML, PDF)

`--css` : CSS personnalisé pour l'export

Options de Debug `--verbose` : Mode verbeux

`--debug` : Mode debug détaillé

`--force` : Forcer l'écrasement

`--validate` : Validation des entrées

📊 Dépendances par Fonctionnalité

Parsing XMI/XML

- **lxml** : Parseur principal XMI/UML
- **xmltodict** : Conversion XML vers dict (optionnel)

Configuration YAML/JSON

- **PyYAML** : Lecture/écriture YAML
- **jsonschema** : Validation JSON/YAML

Interface Utilisateur

- **questionary** : CLI interactive
- **rich** : Interface enrichie (optionnel)

Traitement Données

- **pandas** : Données tabulaires (CSV, Excel)
- **openpyxl** : Support Excel

Validation

- **jsonschema** : Validation schémas
- **pydantic** : Validation données (optionnel)

Export/Output

- **markdown** : Rendering Markdown
- **pdfkit** : Export PDF (optionnel)

🌀 Compatibilité Python

- ✅ Python 3.8+

Systèmes d'exploitation :

- ✅ Windows 10/11
- ✅ Linux
- ✅ macOS 10.15+

Installation

```
# Core seulement
pip install lxml PyYAML questionary jsonschema

# Avec support CSV
pip install lxml PyYAML questionary jsonschema pandas

# Avec visualisation
pip install lxml PyYAML questionary jsonschema matplotlib plotly

# Avec export
pip install markdown==3.4.4 pdfkit==1.0.0 python-docx==1.1.0
pip install markdown pdfkit python-docx

# Installer wkhtmltopdf (requis pour pdfkit)
# Sur Ubuntu/Debian:
sudo apt-get install wkhtmltopdf

# Sur macOS:
brew install wkhtmltopdf

# Sur Windows: Télécharger depuis https://wkhtmltopdf.org/
# Ajouter le chemin d'installation aux variables d'environnement
set PATH=%PATH%;C:\Program Files\wkhtmltopdf\bin

# Vérifier l'installation sur Windows
wkhtmltopdf --version
```

🔍 Vérification Installation

```
# Vérifier les dépendances installées
pip list | grep -E "lxml|PyYAML|questionary|jsonschema"

# Vérifier la version Python
python --version

# Tester l'import des modules
python -c "import lxml; import yaml; import questionary; print('✅ Toutes les
dépendances sont installées')"
```

🔄 Mise à Jour.

```
# Mettre à jour toutes les dépendances
pip install --upgrade -r requirements.txt

# Vérifier les versions obsolètes
pip list --outdated
```

```
# Mettre à jour une dépendance spécifique
pip install --upgrade package_name
```

🔧 Dépannage

Problèmes Courants

- **Erreur : Module non trouvé**

```
# Réinstaller les dépendances
# Installation Complete
pip install -r requirements.txt
```

```
# Installation Minimaliste
pip install -r requirements-minimal.txt
```

```
# Installation avec Options
# Core seulement
pip install lxml PyYAML questionary jsonschema

# Avec support CSV
pip install lxml PyYAML questionary jsonschema pandas

# Avec visualisation
pip install lxml PyYAML questionary jsonschema matplotlib plotly
```

- **Debug Mode** Activez le mode debug pour plus d'informations :

```
python main.py --debug 2> debug.log
```

- **Erreur : Encoding XMI**

```
# Spécifier l'encodage UTF-8
export PYTHONUTF8=1
python main.py
```

- **Fichiers non trouvés**

```
# Vérifier les chemins dans config.json
```

```
python -c "import json; print(json.load(open('config.json'))['paths'])"
```

📋 Bonnes Pratiques

Structure des Données

- **Organisation** : Gardez les fichiers d'entrée dans `input/`
- **Versionnement** : Versionnez les modèles XMI et configurations
- **Sauvegarde** : Backup régulier du dossier `output/`

Qualité de la Documentation

- **Validation** : Utilisez les schémas JSON pour valider les entrées
- **Review** : Revue régulière de la documentation générée
- **Mise à jour** : Maintenance des templates et configurations

Performance

- ⚡ **Génération typique** : 2-10 secondes
- 📄 **Documentation moyenne** : 50-200 pages
- 💾 **Mémoire recommandée** : 8GB+ pour gros modèles

🤝 Contribution

Développement

1. Fork le repository
2. **Feature branch** : `git checkout -b feature/nouvelle-fonctionnalite`
3. **Commit** : `git commit -am 'Ajout nouvelle fonctionnalité'`
4. **Push** : `git push origin feature/nouvelle-fonctionnalite`
5. **Pull Request**

🚀 Aide et Exemples

```
# Afficher l'aide complète
python main.py --help

# Afficher des exemples d'utilisation
python cli_help.py

# Afficher l'aide des sections
python cli_help.py --sections
```

Fonctionnalités DOCX Implémentées

Conversion des Éléments Markdown

- ✅ Titres (H1-H6) avec styles appropriés
- ✅ Paragraphes avec formatage de texte
- ✅ Listes à puces et numérotées
- ✅ Blocs de code avec police monospace
- ✅ Tableaux avec bordures et en-têtes
- ✅ Texte en **gras** et *italique*

Mise en Page Professionnelle

- ✅ En-tête avec titre et date
- ✅ Pied de page avec informations de génération
- ✅ Styles cohérents tout au long du document
- ✅ Sauts de page appropriés

Options Avancées

- ✅ Styles personnalisables via python-docx
- ✅ Métadonnées du document
- ✅ Gestion des erreurs robuste
- ✅ Intégration transparente avec le système existant

🎯 Avantages du DOCX

- **Éditabilité** : Modification facile dans Word
- **Accessibilité** : Format universellement reconnu
- **Collaboration** : Commentaires et révisions
- **Compatibilité** : Office 365, Google Docs, LibreOffice
- **Professionnalisme** : Format standard pour les rapports